2. *Learning Model Parameters*

So far, we have defined how linear models produce predictions from inputs and parameters. We now turn to the question of how the parameters are *learned* from data. For a more detailed treatment of optimisation methods and explicit derivations of the gradients for logistic and multinomial logistic regression, see *Speech and Language Processing* by Jurafsky and Martin (Ed. 3), Chapter 4, in particular Sections 4.4–4.6, 4.8, and 4.15.

Throughout this section, we assume a supervised dataset of N input–output pairs

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}, \tag{77}$$

where $\mathbf{x}^{(i)} \in \mathbb{R}^{d+1}$ is an augmented input representation with a leading constant 1 (i.e., $\mathbf{x}_0^{(i)} = 1$), as introduced above.

We denote by $\boldsymbol{\theta}$ the parameters of the model. For the linear, logistic, and multinomial logistic regression models considered here, $\boldsymbol{\theta}$ corresponds to the weight vector(s) $\mathbf{w}$.[5] Learning consists of solving an optimisation problem of the form

$$\boldsymbol{\theta}^{\star} = \arg\min_{\boldsymbol{\theta}} \mathcal{J}(\boldsymbol{\theta}), \tag{78}$$

where $\mathcal{J}(\boldsymbol{\theta})$ is a dataset-level *objective function* defined in terms of a suitable *loss function*.

[5] We introduce the more general notation to emphasise that the learning framework applies beyond linear models.

Loss Functions and Objective Functions

Learning is formulated as an optimisation problem in which model parameters are chosen to minimise a dataset-level *objective function*. This objective is constructed by aggregating a *loss function* evaluated on individual training examples.

A loss function measures how well the model's prediction for a single example matches the corresponding observed target: it assigns a larger value when predictions are far from the targets and a smaller value when predictions are close. The objective function is obtained by combining the per-example losses over the training

set, typically by taking their average. By adjusting the parameters to reduce the objective function, we can make the model's predictions progressively more accurate.[6]

We now introduce the standard loss functions and corresponding objective functions used for linear regression, logistic regression, and multinomial logistic regression.

[6] Loss minimisation is equivalent to the maximisation of predictive accuracy or likelihood, since such objectives can always be rewritten as the minimisation of an appropriately defined loss (e.g., negative log-likelihood). Both formulations lead to the same parameter values.

Linear Regression: Squared Error

Recall that in linear regression, the prediction for example i is $\widehat{y}^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} \in \mathbb{R}$. A standard loss for linear regression is the *squared error*:

$$\mathcal{L}_{\text{LR}}\left(\widehat{y}^{(i)}, y^{(i)}\right) = \left(\widehat{y}^{(i)} - y^{(i)}\right)^2. \tag{79}$$

This is the squared difference between the model's prediction $\widehat{y}^{(i)}$ and the true target value $y^{(i)}$. It penalises larger errors more strongly than smaller ones, since the deviation is squared, and is symmetric with respect to over- and under-prediction. Minimising the squared error therefore encourages predictions that lie as close as possible to the observed targets in Euclidean distance.[7]

[7] Euclidean distance is the standard notion of geometric distance. In one dimension, it is the absolute difference between two real numbers; in higher dimensions, it is the length of the straight line connecting two points.

At the dataset level, the objective is the mean squared error (MSE),

$$\mathcal{J}_{\text{LR}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_{\text{LR}}\left(\widehat{y}^{(i)}, y^{(i)}; \boldsymbol{\theta}\right). \tag{80}$$

which we have already encountered in the context of model evaluation. The factor $1/N$ does not affect the minimiser, but makes the loss independent of the dataset size.

Logistic Regression: Negative Log-likelihood (Cross-entropy)

In binary logistic regression, the model predicts the probability that an input belongs to the positive class. Let $Y^{(i)}$ denote the binary random variable ($Y^{(i)} \in \{0, 1\}$) associated with the label for input $\mathbf{x}^{(i)}$. The model specifies

$$P\left(Y^{(i)} = 1 \mid \mathbf{x}^{(i)}\right) = \widehat{p}^{(i)} = \sigma\left(\mathbf{w}^\top \mathbf{x}^{(i)}\right), \tag{81}$$

where $\sigma(\cdot)$ is the logistic (sigmoid) function.

A natural learning principle for logistic regression, and for probabilistic models more generally, is *maximum likelihood estimation*: parameters are chosen to assign high probability to the labels observed in the training data. For a single example, logistic regression assumes a Bernoulli distribution over the label.[8] The likelihood of observing the realised label $y^{(i)} \in \{0, 1\}$ is

$$P\left(Y^{(i)} = y^{(i)} \mid \mathbf{x}^{(i)}\right) = \left(\widehat{p}^{(i)}\right)^{y^{(i)}} \left(1 - \widehat{p}^{(i)}\right)^{1 - y^{(i)}}. \tag{82}$$

[8] A Bernoulli distribution models a single binary random variable taking values in $\{0, 1\}$ and is fully characterised by one parameter p, the probability of observing the value 1.

This compact expression covers both possible outcomes of the binary random variable.

In practice, optimisation is performed on the *log-likelihood*, since taking logarithms yields a more numerically stable objective. For a single example,

$$\log P\Big(Y^{(i)} = y^{(i)} \mid \mathbf{x}^{(i)}\Big) = y^{(i)} \log \widehat{p}^{(i)} + \big(1 - y^{(i)}\big) \log\big(1 - \widehat{p}^{(i)}\big). \quad (83)$$

Training proceeds by *minimising* the negative log-likelihood, which defines the per-example loss

$$\mathcal{L}_{\mathrm{NLL}}\Big(\widehat{p}^{(i)}, y^{(i)}\Big) = -\Big[y^{(i)} \log \widehat{p}^{(i)} + \big(1 - y^{(i)}\big) \log\big(1 - \widehat{p}^{(i)}\big)\Big]. \quad (84)$$

This is also known as the *binary cross-entropy loss*.

At the dataset level, the training objective is obtained by averaging the per-example losses,

$$\mathcal{J}_{\mathrm{NLL}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_{\mathrm{NLL}}\Big(\widehat{p}^{(i)}, y^{(i)}; \boldsymbol{\theta}\Big), \quad (85)$$

which is the objective minimised during training.

Multinomial Logistic Regression: Multi-class Cross-entropy

In multinomial logistic regression with K classes, the model predicts a probability distribution over class labels. Let $Y^{(i)}$ denote the categorical random variable associated with the label for input $\mathbf{x}^{(i)}$, taking values in $\{1, \ldots, K\}$. The model specifies

$$P\Big(Y^{(i)} = k \mid \mathbf{x}^{(i)}\Big) = \widehat{p}_k^{(i)}, \qquad k = 1, \ldots, K, \quad (86)$$

where the vector of predicted class probabilities

$$\widehat{\mathbf{p}}^{(i)} = \big(\widehat{p}_1^{(i)}, \ldots, \widehat{p}_K^{(i)}\big) \quad (87)$$

is obtained by applying the softmax function to the class scores.

For a single example, multinomial logistic regression assumes a categorical distribution over the label. If the realised label is $y^{(i)} \in \{1, \ldots, K\}$, the likelihood of the observation is

$$P\Big(Y^{(i)} = y^{(i)} \mid \mathbf{x}^{(i)}\Big) = \widehat{p}_{y^{(i)}}^{(i)}. \quad (88)$$

Using a one-hot encoding $\mathbf{y}^{(i)} \in \{0, 1\}^K$ of the realised label, the per-example negative log-likelihood can be written as

$$\mathcal{L}_{\mathrm{NLL}}\Big(\widehat{\mathbf{p}}^{(i)}, \mathbf{y}^{(i)}\Big) = -\sum_{k=1}^{K} y_k^{(i)} \log \widehat{p}_k^{(i)}, \quad (89)$$

which is the *multi-class cross-entropy loss*. Because $\mathbf{y}^{(i)}$ is one-hot, the sum reduces to the negative log-probability of the correct class, $-\log \hat{p}^{(i)}_{y^{(i)}}$.

At the dataset level, the training objective is

$$\mathcal{J}_{\text{NLL}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_{\text{NLL}}\left(\hat{\mathbf{p}}^{(i)}, \mathbf{y}^{(i)}; \boldsymbol{\theta}\right), \tag{90}$$

which is the objective minimised during training.

Gradient Descent

In most practical settings, the objective function $\mathcal{J}(\boldsymbol{\theta})$ cannot be minimised in closed form and is therefore optimised using iterative methods. One of the most widely used approaches is *gradient descent*, which updates the parameters by moving them in the direction of the steepest decrease of the objective function.

General update rule. Let $\boldsymbol{\theta}$ be a vector collecting all parameters (e.g., $\boldsymbol{\theta} = \mathbf{w}$ for linear or logistic regression; $\boldsymbol{\theta} = W$ for the multinomial case). Given a learning rate $\eta > 0$, gradient descent performs updates

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla_{\boldsymbol{\theta}} \mathcal{J}(\boldsymbol{\theta}). \tag{91}$$

The vector $\nabla_{\boldsymbol{\theta}} \mathcal{J}(\boldsymbol{\theta})$ is the gradient of the objective function with respect to the parameters; it points in the direction of steepest increase of $\mathcal{J}$. Intuitively, the gradient tells us which small change in the parameters would increase the loss the most; gradient descent moves in the opposite direction to reduce it as quickly as possible. The learning rate $\eta > 0$ controls the size of the update: larger values lead to larger steps in parameter space, while smaller values result in more conservative updates. Repeating this update iteratively drives the parameters towards values that locally[9] minimise the objective function. If η is too large, updates may overshoot the minimum and fail to converge; if it is too small, convergence can be very slow.

Generic learning loop. For the models in this chapter, and many other models including neural networks, learning follows the same iterative procedure (or *training loop*):

1. **Predict:** compute model outputs from current parameters.
2. **Compute loss:** evaluate how far predictions are from targets.
3. **Compute gradient:** differentiate the loss w.r.t. parameters.
4. **Update:** move parameters in the negative-gradient direction.

[9] A *local minimum* of an objective function is a parameter setting at which the objective has a lower value than at all sufficiently nearby parameter values, though it may not be the lowest value overall. An objective function is *convex* if it has a simple, bowl-shaped form; intuitively, this means the objective has a single lowest point. For convex objectives, every local minimum is therefore also a global minimum. The objective functions for linear regression (with squared error), logistic regression, and multinomial logistic regression are convex in their parameters, so gradient descent is guaranteed to converge to the global optimum (up to numerical issues), given a suitable learning rate. For non-convex objectives, such guarantees do not hold.

Example 1: Linear regression with intercept only

Consider the simplest linear regression model with only an intercept w_0 and no features:

$$\hat{y}^{(i)} = w_0. \tag{92}$$

The MSE objective is

$$\mathcal{J}(w_0) = \frac{1}{N}\sum_{i=1}^{N}(w_0 - y^{(i)})^2. \tag{93}$$

Although the minimum of $\mathcal{J}(w_0)$ can be computed in closed form as the sample mean, gradient descent reaches the same solution iteratively. The derivative of the MSE objective is

$$\frac{\partial \mathcal{J}}{\partial w_0} = \frac{2}{N}\sum_{i=1}^{N}(w_0 - y^{(i)}). \tag{94}$$

Gradient descent therefore updates

$$w_0 \leftarrow w_0 - \eta\,\frac{2}{N}\sum_{i=1}^{N}(w_0 - y^{(i)}). \tag{95}$$

This update pushes w_0 towards the sample mean of the targets, as can be seen in Figure 3.

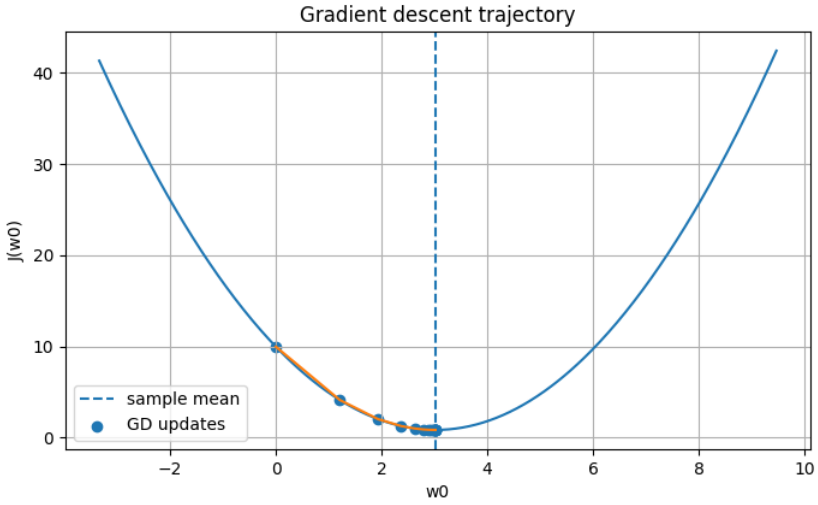

Figure 3: Gradient descent trajectory on the MSE objective for an intercept-only linear regression model. The x-axis shows the model parameter w_0 and the y-axis shows the MSE objective $\mathcal{J}(w_0)$. The vertical dashed line indicates the analytical minimum of the objective, corresponding to the sample mean of the target values. The scatter points mark successive gradient descent updates, and the orange line traces the optimisation path from the initial value of w_0 towards the minimum. Gradient descent updates w_0 in the direction opposite to the gradient, monotonically reducing the MSE and converging to the optimal solution.

Example 2: One feature plus intercept (2D parameter space)

Now consider a single feature $x^{(i)}$ and parameters (w_0, w_1):

$$\hat{y}^{(i)} = w_0 + w_1 x^{(i)}. \tag{96}$$

The parameter vector $\boldsymbol{\theta} = (w_0, w_1)$ consists of two weights, so the objective is a function of two variables:

$$\mathcal{J}(w_0, w_1) = \frac{1}{N} \sum_{i=1}^{N} (w_0 + w_1 x^{(i)} - y^{(i)})^2. \tag{97}$$

Because $\mathcal{J}$ depends on more than one variable, we use the partial derivatives $\frac{\partial \mathcal{J}}{\partial w_0}$ and $\frac{\partial \mathcal{J}}{\partial w_1}$ to isolate the effect of changing a single parameter on the value of the objective. The gradient of $\mathcal{J}$ is therefore a two-dimensional vector:

$$\nabla \mathcal{J}(w_0, w_1) = \begin{pmatrix} \frac{\partial \mathcal{J}}{\partial w_0} \\ \frac{\partial \mathcal{J}}{\partial w_1} \end{pmatrix}, \tag{98}$$

where the component $\frac{\partial \mathcal{J}}{\partial w_0}$ captures how the objective varies when the intercept is adjusted, and the component $\frac{\partial \mathcal{J}}{\partial w_1}$ captures how the objective varies when the slope of the prediction function with respect to x is changed. The gradient vector $\nabla \mathcal{J}(w_0, w_1)$ thus points in the direction of steepest increase of the surface $\mathcal{J}(w_0, w_1)$ in the two-dimensional parameter space. Gradient descent updates both parameters simultaneously by moving in the opposite direction, i.e., towards the minimum of the objective:

$$w_0 \leftarrow w_0 - \eta \frac{\partial \mathcal{J}}{\partial w_0}, \qquad w_1 \leftarrow w_1 - \eta \frac{\partial \mathcal{J}}{\partial w_1}. \tag{99}$$

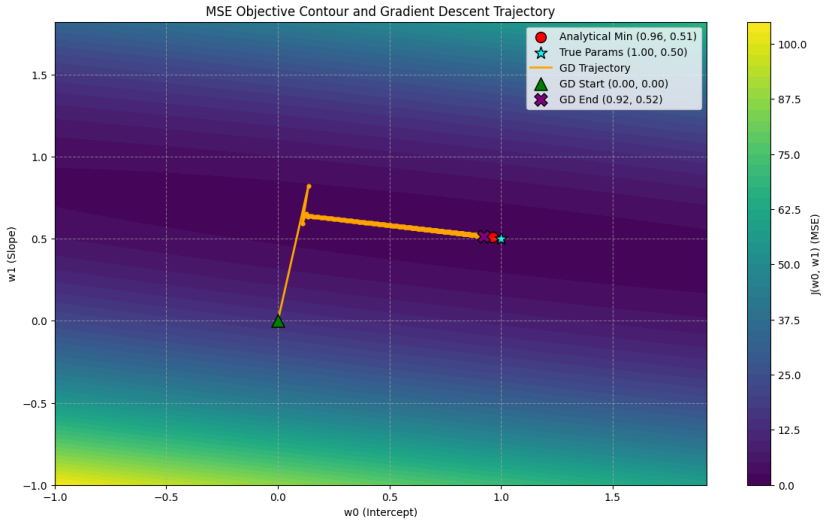

Figure 4: Gradient descent trajectory on the MSE objective for linear regression in a two-dimensional parameter space. Contours show value areas of $\mathcal{J}(w_0, w_1)$, with the minimum at the centre. The analytical solution for an example, randomly-generated dataset is marked in red; the ground-truth parameters used to generate the data are shown in cyan. The orange line traces the gradient descent updates from the initial point $(w_0, w_1) = (0, 0)$ to the final parameters after 300 steps with learning rate $\eta = 0.02$.

Stochastic Gradient Descent

In the previous section, gradient descent was described as computing the gradient of the objective function using the entire training dataset at each update, which is also commonly referred to as *batch gradient descent*. Batch gradient descent can be computationally expensive when the dataset is large and is infeasible in online or streaming settings, where data are processed sequentially. An alternative and widely used optimisation method is *stochastic gradient descent* (SGD).

Instead of computing the gradient using all N training examples, stochastic gradient descent approximates the true gradient using a single training example (or, more generally, a small subset of examples). This greatly reduces the computational cost of each update and allows learning to scale to large datasets.

Update rule. Let $(x^{(i)}, y^{(i)})$ denote a single training example sampled from the dataset. Given a learning rate $\eta > 0$, stochastic gradient descent performs updates of the form

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla_{\boldsymbol{\theta}} \mathcal{L}\Big(x^{(i)}, y^{(i)}; \boldsymbol{\theta}\Big), \tag{100}$$

where $\mathcal{L}(\boldsymbol{\theta}; x^{(i)}, y^{(i)})$ is the loss incurred on that single example. The update is repeated for many iterations, typically cycling through the dataset multiple times. The gradient used in SGD is a *noisy but unbiased estimate* of the true gradient computed over the full dataset, meaning that although individual updates may not move directly towards the minimum (noisy), their average effect does (unbiased).

Mini-batch gradient descent. In practice, SGD is often implemented using small batches of B examples, known as *mini-batches*. For a mini-batch $\mathcal{B}$, the update becomes

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \frac{1}{|\mathcal{B}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \mathcal{L}\Big(x^{(i)}, y^{(i)}; \boldsymbol{\theta}\Big). \tag{101}$$

Mini-batch gradient descent offers a compromise between the stability of batch gradient descent and the efficiency of one-example stochastic gradient descent, and is the standard choice in modern machine learning.